



Instituto Politécnico do Cávado e do Ave

Escola Superior De Tecnologia

Licenciatura em Engenharia de Sistemas Informáticos

UC: Estrutura de Dados Avançadas

Relatório de Trabalho Prático – Fase 1

Aluno: Rúben Manuel Abreu de Oliveira, a24861

Docente Responsável: Luis Gonzaga Martins Ferreira

Ano Letivo: 2024/2025

Repositório GitHub: <https://github.com/RubenO94/EDA-TP/tree/v-1-0>

Março, 2025

Resumo (versão preliminar com base na Fase 1)

O presente relatório descreve o desenvolvimento de uma estrutura de dados dinâmica, implementada em linguagem C, com o objetivo de aplicar algoritmos fundamentais abordados durante as aulas da unidade curricular de Estruturas de Dados Avançadas. Nesta primeira fase, o foco principal recaiu sobre a criação, inserção, pesquisa, remoção e libertação de elementos de uma lista ligada.

Embora os algoritmos base tenham sido definidos no âmbito dos conteúdos lecionados, a lógica de negócio associada à estrutura — nomeadamente a representação de antenas e dos efeitos de interferência entre elas — foi deixada à liberdade de implementação do aluno, permitindo a exploração criativa de soluções alinhadas com os requisitos propostos. A abordagem adotada, bem como as decisões de modelação, serão detalhadamente explicadas ao longo do presente relatório.

Este trabalho visa consolidar os conhecimentos práticos sobre listas ligadas, reforçar o raciocínio algorítmico e promover boas práticas de programação em linguagem C, aliando a vertente técnica à resolução de um problema contextualizado

Tabela de Conteúdo

Capítulo 1 – Introdução	1
Motivação	2
Enquadramento	2
Objetivos	2
Metodologias de Trabalho	3
Plano de Trabalho	4
Estrutura do Documento	5
Capítulo 2 - Revisão da Literatura e Enquadramento Teórico	7
Estruturas de Dados Dinâmicas	7
Gestão de Memória em Linguagem C	8
Documentação com Doxygen	9
Ferramentas Utilizadas	10
Capítulo 3 – Análise e Especificação	11
Requisitos	11
Requisitos Funcionais	11
Requisitos Não Funcionais	12
Regras de Negócio e Validações	12
R1 – Posição Única por Antena	12
R2 – Frequência Case-Insensitive	13
R3 – Identificação de Interferência	13
R4 – Representação de Efeitos Nefastos	13
Arquitetura Lógica	14

Organização Modular do Código.....	15
Fluxo de Execução da Aplicação.....	16
Interação e Fluxo de Operações.....	17
Capítulo 4 – Implementação	18
Estrutura Geral do Projeto	18
Principais Funcionalidades Implementadas.....	19
Boas Práticas e Organização (<i>reforçada</i>)	20
Capítulo 5 – Análise e Discussão de Resultados	21
Decisões Técnicas Justificadas	21
Dificuldades Técnicas Sentidas	21
Falhas Identificadas e Aprendizagens	22
Lições Aprendidas e Melhoria Contínua	23
Propostas de Melhoria Futuras	23
Utilização de apontador tail	23
Reestruturação de funções com lógica repetida.....	23
Generalização com apontadores para função.....	24
Capítulo 6 – Conclusão.....	25
Referências	26

Capítulo 1 – Introdução

O presente capítulo pretende apresentar o contexto e os objetivos do trabalho desenvolvido no âmbito da unidade curricular de Estruturas de Dados Avançadas. Para tal, começa-se por explicar a motivação para a realização deste projeto, identificando a sua relevância prática e académica. Em seguida, é descrito o enquadramento do trabalho no percurso académico do aluno, os objetivos propostos, a metodologia adotada para o seu desenvolvimento e o plano de trabalho. Por fim, é apresentada a estrutura do relatório.

Motivação

A motivação para o desenvolvimento deste trabalho está diretamente relacionada com a necessidade de consolidar conhecimentos adquiridos na unidade curricular de Estruturas de Dados Avançadas, particularmente no que diz respeito à implementação e manipulação de listas ligadas em linguagem C.

As listas ligadas constituem uma das estruturas dinâmicas fundamentais no estudo da informática, permitindo uma gestão eficiente de memória e dados em cenários onde a dimensão dos conjuntos de elementos não é conhecida à partida. A sua correta implementação exige domínio de apontadores, alocação e libertação dinâmica de memória, bem como uma organização lógica e modular do código.

Enquadramento

Este trabalho foi desenvolvido no âmbito da unidade curricular de Estruturas de Dados Avançadas, pertencente ao primeiro ano da Licenciatura em Engenharia de Sistemas Informáticos, lecionada no Instituto Politécnico do Cávado e do Ave (IPCA).

O projeto aqui documentado corresponde à Fase 1 do trabalho prático proposto, consistindo no desenvolvimento de uma aplicação que implementa uma estrutura dinâmica para representar um conjunto de antenas e efeitos de interferência, simulando uma situação real com regras específicas de negócio.

Este trabalho representa uma componente essencial do processo de aprendizagem, contribuindo para a consolidação dos conhecimentos teóricos e para o desenvolvimento de competências práticas fundamentais no contexto da programação em baixo nível e gestão de memória dinâmica.

Objetivos

Os principais objetivos definidos para esta fase do trabalho prático são os seguintes:

- Aplicar os algoritmos fundamentais estudados em aula, nomeadamente:
 - criação de nós dinamicamente
 - inserção ordenada
 - pesquisa
 - remoção
 - libertação de memória em listas ligadas.
- Implementar uma estrutura dinâmica capaz de representar antenas posicionadas num plano cartesiano, com base em coordenadas x e y e uma frequência associada.
- Definir e aplicar uma lógica de negócio que permita detetar automaticamente interferências entre antenas com a mesma frequência, quando a distância entre elas for inferior ao valor mínimo permitido.
- Representar graficamente ou simbolicamente os efeitos nefastos resultantes da interferência, através da criação de nós especiais na lista.
- Garantir a modularidade e legibilidade do código, promovendo boas práticas de programação em linguagem C.
- Preparar a estrutura base para futuras fases do projeto, nomeadamente a leitura e escrita de ficheiros com versões do estado da estrutura.

Metodologias de Trabalho

A metodologia de trabalho adotada nesta primeira fase do projeto assentou numa abordagem iterativa e incremental, permitindo um desenvolvimento progressivo da estrutura e das funcionalidades pretendidas. O trabalho foi organizado em várias etapas, iniciando-se com a análise dos requisitos funcionais, seguida pela modelação da estrutura de dados principal e posterior implementação das operações sobre a lista ligada.

Numa primeira fase, foram implementadas as operações base ensinadas em aula, nomeadamente: criação de nós, inserção no início ou noutras posições, pesquisa por coordenadas, remoção de elementos e libertação de memória. A seguir, foi

desenhada e aplicada a lógica de negócio, que define regras para a validação de posições, detecção de interferência e inserção de efeitos nefastos.

O desenvolvimento foi feito com foco na organização modular do código, separando as diferentes responsabilidades por ficheiros e funções, de forma a facilitar a leitura, manutenção e extensão futura da aplicação. Paralelamente, foram realizados testes manuais para validar o comportamento correto da estrutura em diferentes cenários, assegurando o cumprimento das regras definidas e a integridade da memória.

Durante o desenvolvimento, recorreu-se também ao uso da ferramenta Doxygen para documentar o código-fonte de forma clara e estruturada. Esta ferramenta permitiu gerar automaticamente uma documentação técnica das funções, estruturas e ficheiros implementados, facilitando a compreensão da lógica do programa e promovendo boas práticas de programação. A documentação foi mantida atualizada ao longo do projeto, refletindo as alterações e melhorias realizadas durante o processo de desenvolvimento.

Plano de Trabalho

O enunciado da Fase 1 do trabalho prático foi disponibilizado no dia **14 de março de 2025**, com data de entrega estipulada para **30 de março de 2025**. Dado o tempo reduzido para o desenvolvimento completo e documentação detalhada, o foco inicial recaiu sobre a implementação funcional da estrutura de dados e da lógica de negócio, com testes manuais para validar o comportamento do sistema.

Na data prevista, foi entregue **um relatório preliminar em formato Markdown**, que cobria os aspetos essenciais do trabalho, mas com uma descrição sucinta e sem o rigor técnico e formal exigido num relatório académico completo. Por esse motivo, e com o objetivo de melhorar a qualidade da documentação, decidiu-se posteriormente reestruturar e reescrever o relatório, seguindo as normas de redação técnica e científica recomendadas.

O novo plano de trabalho inclui, além da revisão do código e organização por módulos, a escrita estruturada do relatório com base nas secções definidas, a utilização da ferramenta Doxygen para a criação da documentação técnica e a preparação de anexos de apoio.

A distribuição do tempo ficou, assim, organizada da seguinte forma:

- **14 a 29 de março:**
 - Implementação das operações sobre listas ligadas
 - Desenvolvimento da lógica de negócio (frequência, interferências, efeitos)
 - Documentação técnica com Doxygen
- **29 a 30 de março:**
 - Testes manuais e entrega de relatório simplificado (formato Markdown)
- **Abril de 2025:**
 - Reescrita completa do relatório, em formato académico e com maior profundidade

Estrutura do Documento

Este relatório está organizado em seis capítulos, para além da presente introdução. A estrutura adotada visa apresentar de forma clara e progressiva todo o trabalho desenvolvido, desde a fundamentação teórica até à análise dos resultados obtidos.

No **Capítulo 2** é realizada uma revisão dos principais conceitos teóricos e técnicos relevantes para a implementação do projeto, incluindo estruturas de dados dinâmicas, listas ligadas e manipulação de memória em linguagem C.

O **Capítulo 3** apresenta a análise do problema e a especificação da solução, incluindo os requisitos definidos, a lógica de negócio associada às regras de interferência entre antenas, e a modelação da estrutura de dados desenvolvida.

No **Capítulo 4**, é detalhada a implementação da aplicação, descrevendo-se a organização dos módulos, as principais funções desenvolvidas e as decisões técnicas tomadas durante o processo de desenvolvimento.

O **Capítulo 5** foca-se na análise e discussão dos resultados obtidos, apresentando casos de teste relevantes, validando o comportamento do sistema e refletindo sobre as suas limitações e possíveis melhorias.

Por fim, o **Capítulo 6** apresenta as conclusões do trabalho, destacando os objetivos atingidos, as dificuldades encontradas e as propostas para trabalhos futuros.

O relatório inclui ainda uma secção de **Referências Bibliográficas**, com todas as fontes consultadas, bem como anexos com documentação de apoio e exemplos práticos da estrutura desenvolvida

Capítulo 2 - Revisão da Literatura e Enquadramento Teórico

O presente capítulo tem como objetivo apresentar os principais conceitos teóricos e técnicos que sustentam o trabalho desenvolvido. Para isso, são abordadas estruturas de dados dinâmicas com foco nas listas ligadas, bem como técnicas de gestão de memória em linguagem C. Adicionalmente, são exploradas as ferramentas utilizadas para apoiar o desenvolvimento e documentação do projeto, e, por fim, são referidas algumas abordagens semelhantes ou comparáveis, com uma breve análise comparativa.

Estruturas de Dados Dinâmicas

As estruturas de dados dinâmicas desempenham um papel essencial no desenvolvimento de aplicações que exigem flexibilidade no armazenamento e manipulação de dados. Ao contrário das estruturas estáticas, como os arrays, as estruturas dinâmicas permitem que a memória seja alocada e libertada durante a execução do programa, ajustando-se às necessidades reais do sistema em tempo de execução(Weiss, 2011).

Entre as estruturas dinâmicas mais utilizadas, as **listas ligadas** (ou linked lists) destacam-se pela sua simplicidade e eficiência em operações como inserção e remoção de elementos. Numa lista ligada, cada nó contém os dados relevantes e um apontador para o próximo nó da sequência, o que elimina a necessidade de mover elementos ao inserir ou remover (Horowitz et al., 2009). Esta característica torna as listas ideais para cenários onde a dimensão dos dados não é conhecida previamente ou pode variar com frequência.

A utilização de listas ligadas é especialmente recomendada quando se pretende manter um conjunto de elementos ordenados com base em critérios dinâmicos, como ocorre no projeto aqui descrito, onde as antenas são organizadas por coordenadas. A abordagem adotada baseia-se numa lista simplesmente ligada, suficiente para percorrer sequencialmente os elementos, verificar interferências entre antenas e representar efeitos nefastos. Esta escolha foi influenciada pela clareza de implementação e pela eficiência em memória, aspetos amplamente discutidos na literatura (Knuth, 1997).

Gestão de Memória em Linguagem C

A linguagem C oferece ao programador um controlo total sobre a alocação e libertação de memória, o que constitui simultaneamente uma vantagem e uma responsabilidade. Ao contrário de linguagens modernas com gestão automática de memória (garbage collection), em C é necessário recorrer explicitamente a funções como *malloc* e *free*, todas incluídas na biblioteca *<malloc.h>* (Damas, 2002).

A correta gestão de memória é crucial para garantir a estabilidade, desempenho e fiabilidade de aplicações em C. Erros nesta área podem resultar em **memory leaks**, **acessos inválidos a memória libertada** (dangling pointers) ou ainda **corrupção de memória**, problemas que, embora comuns, são muitas vezes difíceis de detetar durante a execução normal do programa (Weiss, 2011).

No âmbito do presente projeto, a gestão de memória foi uma das componentes centrais, uma vez que a estrutura principal — uma lista simplesmente ligada — exige que cada nó seja alocado dinamicamente no momento da inserção, e libertado corretamente aquando da sua remoção ou destruição da lista. Para tal, recorreu-se à separação clara entre as funções de criação de nós e as de inserção, promovendo o encapsulamento das responsabilidades e reduzindo o risco de falhas associadas à manipulação de apontadores.

A necessidade de libertar todos os nós antes de terminar a execução do programa levou à implementação de uma função de destruição completa da lista, que percorre todos os elementos e liberta a memória associada a cada um. Esta prática segue as recomendações de boas práticas em programação em C, que defendem que qualquer memória alocada deve, sem exceção, ser libertada antes do encerramento do programa (Horowitz et al., 2009).

Documentação com Doxygen

Uma das boas práticas na engenharia de software é a manutenção de documentação técnica clara e atualizada, especialmente em projetos desenvolvidos em linguagens como C, onde a legibilidade e a organização do código nem sempre são garantidas pela própria linguagem. Para este fim, foi utilizada a ferramenta **Doxygen**, uma aplicação amplamente adotada para a **criação automática de documentação técnica** a partir de comentários inseridos diretamente no código-fonte.

O Doxygen permite gerar documentação em diversos formatos (HTML, PDF, RTF, LaTeX, entre outros) e suporta linguagens como C, C++, Java, Python, entre outras. No contexto do presente projeto, foi utilizado para produzir documentação estruturada do código desenvolvido em C, incluindo listas de funções, estruturas de dados, diagramas de dependência, e explicações associadas a cada componente do sistema.

A utilização do Doxygen ao longo do desenvolvimento revelou-se fundamental para manter uma visão clara da estrutura do código, facilitando a sua organização e manutenção. Para tal, foram utilizados blocos de comentários com marcações especiais (como `/** */`, `@param`, `@return`, etc.), permitindo que o Doxygen extraísse automaticamente a informação necessária para compor a documentação.

Esta abordagem não só melhorou a legibilidade e navegabilidade do código, como também promoveu o rigor na escrita de funções bem definidas e devidamente comentadas, seguindo os princípios recomendados por autores como (Martin, 2008),

que defende que “*código limpo é aquele que foi bem escrito e bem pensado*”, e que a documentação deve complementar — e não substituir — a clareza do código em si.

Ferramentas Utilizadas

O desenvolvimento do projeto foi suportado por um conjunto de ferramentas que permitiram implementar, testar e documentar a solução de forma eficiente e estruturada.

Para a edição e organização do código, foi utilizado o editor **Visual Studio Code**, com suporte para extensões de realce de sintaxe, e integração com o terminal. A compilação e execução do programa foram feitas num ambiente Linux, através da utilização do compilador **GCC (GNU Compiler Collection)**. Esta escolha deveu-se à sua fiabilidade, ao suporte de padrões modernos da linguagem C e à facilidade de automação de tarefas de compilação através de **Makefiles**.

O controlo de versões foi realizado localmente com **Git**, de forma a manter o histórico de alterações e possibilitar a reversão de estados anteriores do código em caso de erro. A utilização do Git revelou-se útil, mesmo sendo um projeto individual, na organização das fases de implementação.

A documentação técnica do código foi produzida com **Doxygen**, como referido na secção anterior, permitindo criar documentação técnica e manter o código comentado com clareza. Para a redação do relatório final, foi utilizada a aplicação **Microsoft Word**, com formatação de acordo com as normas académicas indicadas pela unidade curricular.

Capítulo 3 – Análise e Especificação

O presente capítulo tem como objetivo apresentar a análise do problema abordado neste projeto, bem como a especificação técnica da solução proposta. São identificados os requisitos funcionais e as principais regras de negócio que regem o comportamento da aplicação, e é feita uma descrição da arquitetura lógica adotada. A modelação das estruturas de dados e a forma como estas interagem entre si são também abordadas nesta secção, preparando o leitor para a compreensão detalhada da implementação apresentada no capítulo seguinte.

Requisitos

Requisitos Funcionais

Os requisitos funcionais definem as funcionalidades mínimas que o sistema deve implementar para cumprir os objetivos definidos:

- **RF1** – Permitir a criação de uma estrutura dinâmica do tipo lista ligada para armazenar os elementos.
- **RF2** – Inserir novos elementos (antenas) na lista, com coordenadas x e y e uma frequência associada.
- **RF3** – Verificar se a posição de inserção já está ocupada por outra antena.
- **RF4** – Identificar automaticamente situações de interferência entre antenas da mesma frequência, se a distância entre elas for inferior ao valor definido como mínimo aceitável.
- **RF5** – Registar os efeitos nefastos da interferência, através da criação de novos elementos (com a frequência #) nas posições afetadas.
- **RF6** – Permitir remover elementos da lista, incluindo antenas e efeitos nefastos.
- **RF7** – Permitir a pesquisa de elementos com base nas coordenadas.
- **RF8** – Libertar corretamente toda a memória alocada, destruindo a lista no fim da execução.

- **RF9** – Organizar o código de forma modular, com separação clara entre criação de nós, inserção, validação, remoção e impressão da estrutura.

Requisitos Não Funcionais

Além das funcionalidades, foram também definidos alguns critérios de qualidade a observar:

- **RNF1** – O código deve ser escrito em linguagem C.
- **RNF2** – O código deve seguir boas práticas de legibilidade, modularidade e gestão de memória.
- **RNF3** – A documentação técnica do código deve ser gerada automaticamente com Doxygen.
- **RNF4** – O sistema deve permitir fácil manutenção e expansão para fases futuras.

Regras de Negócio e Validações

O funcionamento da estrutura desenvolvida baseia-se num conjunto de regras de negócio que garantem a integridade dos dados e a coerência do sistema. Estas regras foram definidas com base no enunciado da fase 1 do projeto, complementadas por decisões tomadas pelo aluno durante a análise e implementação.

R1 – Posição Única por Antena

Não é permitida a existência de duas antenas na mesma posição. Assim, ao tentar inserir uma antena numa coordenada (x, y) já ocupada por outra antena, a inserção é rejeitada. Esta validação evita duplicações e garante que cada antena representa uma localização única no plano.

R2 – Frequência Case-Insensitive

A frequência de cada antena é representada por um caractere do alfabeto. No entanto, para efeitos de comparação, a frequência é considerada **insensível a maiúsculas e minúsculas** (ex: **A** e **a** são tratadas como iguais). Esta decisão foi adotada para simplificar a lógica de interferência e evitar duplicações implícitas.

R3 – Identificação de Interferência

Sempre que duas ou mais antenas com a **mesma frequência** são inseridas e a **distância entre elas for inferior a N unidades**, considera-se que há interferência. A distância é calculada com base na **distância Euclidiana** entre os pontos:

$$\text{distância} = \sqrt{\{(x_2 - x_1)^2 + (y_2 - y_1)^2\}}$$

Se a distância for inferior ao valor mínimo aceitável (N), o sistema deteta automaticamente a interferência e prossegue com a criação de efeitos nefastos.

R4 – Representação de Efeitos Nefastos

A interferência detetada dá origem à criação de elementos do tipo **efeito nefasto**, representados com o caractere **#** como frequência. Os efeitos nefastos são inseridos automaticamente em todas as posições **entre as antenas em conflito**, respeitando a simetria e propagação do efeito.

- Se a interferência for a 2 unidades de distância, o efeito propaga-se 4 posições entre as duas antenas.
- A regra geral adotada foi: **interferência** $\rightarrow 2 \times$ (**distância entre antenas**) **posições afetadas**

Os efeitos nefastos podem ser criados mesmo em posições já ocupadas por outras antenas ou por efeitos anteriores, sendo o símbolo **#** considerado prioritário para marcar zonas de conflito.

Arquitetura Lógica

A aplicação foi desenvolvida com uma arquitetura modular, baseada na separação de responsabilidades por ficheiros e funções específicas. Esta abordagem permite melhorar a legibilidade, facilitar a manutenção do código e preparar a base para possíveis evoluções do sistema em fases futuras.

O núcleo da aplicação assenta em duas listas ligadas distintas:

- Uma lista para armazenar **antenas** (com coordenadas e frequência)
- Uma lista separada para armazenar **efeitos nefastos** causados por interferência

Ambas as listas utilizam a **mesma estrutura de dados (struct)**, que contém os campos:

```
typedef struct ED
{
    char frequency;
    int x, y;
    struct ED *next;
} ED;
```

Esta decisão permite **reutilizar as funções de criação, pesquisa e remoção**, mantendo uma lógica comum para o tratamento de elementos no plano, enquanto se distingue claramente entre tipos de elementos através do conteúdo da frequência ou pela lista onde estão inseridos.

Organização Modular do Código

A seguir apresenta-se um resumo dos módulos existentes e das suas responsabilidades:

- **ed.c / ed.h**

Contém as funções principais de manipulação da lista ligada (criação, inserção, remoção, pesquisa, destruição), comuns tanto a antenas como a efeitos nefastos, visto que ambas as listas usam a mesma struct.

- **ed_validator.c / ed_validator.h**

Responsável pelas validações de posições e frequências, incluindo detecção de duplicações, limites do plano e consistência dos dados inseridos.

- **geometry.c / geometry.h**

Implementa funções matemáticas auxiliares, como o cálculo da **distância Euclidiana**, central para a lógica de interferência.

- **effect.c / effect.h**

Contém a lógica para criação e inserção de efeitos nefastos (frequency = '#'), com base na distância entre antenas da mesma frequência.

- **controller.c / controller.h**

Responsável pela coordenação geral das operações entre módulos (ex: inserção de antenas, verificação de regras e gestão das listas).

- **display.c / display.h**

Contém as funções de visualização da estrutura no terminal, com suporte a **cores** através do módulo terminal_colors.

- **menu.c / menu.h**

Define a interface textual com o utilizador, com base em menus e opções interativas.

- **ed_io.c / ed_io.h**

Prevê a leitura e escrita de ficheiros para persistência de dados (a ser explorado em fases futuras).

- **constants.h**

Centraliza constantes globais utilizadas pelos vários módulos, como limites de distância, dimensões do plano, etc.

- **terminal_colors.h**

Define macros e strings ANSI para a aplicação de cores no terminal, melhorando a visualização de elementos.

Fluxo de Execução da Aplicação

O ponto de entrada da aplicação é o ficheiro **main.c**, que contém a função *main()*. Nesta função, é invocada a função *start_menu()*, definida em *menu.c*, responsável por apresentar o menu principal e gerir a interação com o utilizador.

O módulo *menu.c* está encarregue da navegação pelas diferentes opções da aplicação. Para executar as funcionalidades escolhidas pelo utilizador (como inserir uma antenna, listar elementos, remover posições, etc.), este módulo invoca diretamente funções definidas no módulo *controller.c*.

O ficheiro *controller.c* atua como **orquestrador central** do sistema. Através das funções declaradas em *controller.h*, este módulo articula a comunicação entre os diversos componentes especializados: manipulação de listas (*ed.c*), validações (*ed_validator.c*), lógica geométrica (*geometry.c*), criação de efeitos nefastos (*effect.c*) e visualização (*display.c*).

Esta organização promove uma arquitetura em camadas, onde a interação com o utilizador é separada da lógica interna da aplicação, e onde cada módulo tem uma responsabilidade bem definida. Tal abordagem facilita a manutenção do sistema e o isolamento de erros, alinhando-se com boas práticas de desenvolvimento modular (Martin, 2008).

Interação e Fluxo de Operações

A aplicação foi desenvolvida com base numa interface textual simples, permitindo ao utilizador inserir antenas, visualizar a estrutura, remover elementos e sair do programa. Ao inserir uma antena, o sistema valida a posição, compara a frequência com as existentes e, se detetada interferência, gera automaticamente os efeitos nefastos. Todas as operações são coordenadas por um menu principal e conduzidas por um controlador central, que comunica com os módulos responsáveis por cada funcionalidade.

Capítulo 4 – Implementação

O presente capítulo descreve a implementação do sistema desenvolvido, com base na análise e especificações apresentadas anteriormente. São abordadas as principais decisões técnicas, a estrutura geral do código, as funcionalidades implementadas, bem como algumas boas práticas aplicadas ao longo do desenvolvimento. Pretende-se com isto dar ao leitor uma visão clara sobre a forma como o projeto foi construído em termos práticos.

Estrutura Geral do Projeto

A aplicação foi desenvolvida em linguagem C, com base numa arquitetura modular. O código encontra-se dividido em múltiplos ficheiros .c e .h, organizados nos diretórios:

- **src/** – implementação das funcionalidades
- **include/** – cabeçalhos com definições e assinaturas
- **docs/** – relatório e documentação gerada automaticamente pelo Doxygen
- **input/** – ficheiros auxiliares de entrada

Cada módulo lida com uma responsabilidade específica. Por exemplo, o módulo `ed.c` trata da gestão de listas ligadas, enquanto o `geometry.c` trata do cálculo de distâncias. Esta organização modular permitiu manter o código limpo, reutilizável e facilmente navegável.

Decidiu-se manter a estrutura do projeto separada por diretórios, pois esta abordagem facilita a manutenção, promove a organização e permite localizar rapidamente funções específicas, algo que se tornaria essencial em fases futuras mais complexas.

Principais Funcionalidades Implementadas

Entre as funcionalidades desenvolvidas destacam-se:

- Criação de elementos dinâmicos, com alocação de memória controlada
- Inserção de antenas, com verificação de posições já ocupadas
- Cálculo da distância Euclidiana entre antenas para detecção de interferência
- Criação automática de efeitos nefastos (#) em zonas de conflito
- Remoção de elementos (antenas e efeitos) por coordenadas
- Libertação de memória no encerramento da aplicação
- Interface textual baseada em menus
- Visualização gráfica no terminal, com uso de cores

A inserção dos elementos na lista foi implementada com um algoritmo que mantém a lista ordenada com base nas coordenadas, de forma a facilitar a leitura e garantir um comportamento previsível. Esta decisão visou também preparar o projeto para futuras funcionalidades que beneficiem da ordenação.

Optou-se por isolar a lógica de criação de nós da lógica de inserção, mantendo as funções com responsabilidades únicas, conforme as boas práticas de desenvolvimento modular. Cada função foi projetada para executar apenas uma tarefa clara, como validar, criar, inserir ou verificar conflitos.

Boas Práticas e Organização *(reforçada)*

Durante o desenvolvimento foram seguidas boas práticas:

- Funções pequenas e coesas, cada uma com uma única responsabilidade
- Separação clara entre criação, validação, inserção e visualização
- Modularização do código por ficheiros
- Nomes de variáveis e funções em inglês, legíveis e consistentes
- Uso de constantes globais centralizadas
- Comentários estruturados compatíveis com Doxygen

Evitaram-se funções com múltiplas responsabilidades, seguindo o princípio da responsabilidade única (SRP). Por exemplo, a função de inserção apenas insere o nó ordenadamente; a validação da posição foi feita noutra função antes da chamada. Esta abordagem contribuiu para um código mais fácil de testar e manter.

Capítulo 5 – Análise e Discussão de Resultados

Este capítulo apresenta uma reflexão crítica sobre o trabalho desenvolvido, tendo por base as decisões técnicas tomadas, os desafios encontrados ao longo da implementação, as falhas identificadas e as oportunidades de melhoria detetadas. A análise tem como objetivo evidenciar a capacidade de identificar fragilidades no código, justificar escolhas técnicas e reconhecer o processo de aprendizagem envolvido.

Decisões Técnicas Justificadas

Durante a implementação, foram tomadas decisões com base na clareza, eficiência e modularidade do código. Entre elas destacam-se:

- **Separação entre listas de antenas e efeitos nefastos**, permitindo tratamento independente e facilitando a visualização e limpeza das estruturas.
- **Inserção ordenada por coordenadas**, adotada para melhorar a organização visual e lógica da estrutura, mesmo que isso implicasse maior complexidade na implementação.
- **Comparação case-insensitive de frequências**, para evitar conflitos implícitos e simplificar a verificação de interferência.
- **Separação clara de responsabilidades entre funções**, garantindo que cada função execute apenas uma tarefa (por exemplo, *create_ed()* apenas cria nós, enquanto a verificação é feita antes da inserção).

Estas decisões visaram melhorar a manutenção do sistema, bem como alinhar a estrutura do código com boas práticas de engenharia de software.

Dificuldades Técnicas Sentidas

Durante o desenvolvimento surgiram diversos desafios, sobretudo relacionados com listas ligadas e propagação de interferências:

- **Gestão correta de apontadores em inserção ordenada**, especialmente no tratamento de inserção no início, meio ou fim da lista, exigiu testes cuidadosos e depuração intensiva.
- **Lógica de propagação de efeitos nefastos**, que envolvia criar múltiplos nós em posições calculadas entre duas antenas, revelou-se mais complexa do que inicialmente previsto.
- **Visualização no terminal**, com distinção entre antenas e interferência, exigiu repetidos testes até atingir uma representação clara e compreensível.

Apesar destas dificuldades, todas as funcionalidades fundamentais foram implementadas com sucesso.

Falhas Identificadas e Aprendizagens

A análise posterior ao projeto, em especial após validação com o docente, revelou falhas e aspetos que podem ser otimizados:

- **Repetição de lógica na função `remove_ed()`**, que reimplementa a pesquisa por coordenadas em vez de reutilizar a função `find_ed()`. Esta duplicação contraria o princípio de reutilização de código e poderá ser corrigida com refatoração.
- **Ausência de validação de limites da matriz nas funções de inserção e remoção**, o que permite que coordenadas inválidas (fora do plano) sejam inseridas ou removidas sem verificação. Esta validação deveria ter sido centralizada numa função no módulo de validação.
- **Libertação silenciosa de memória**, sem qualquer verificação durante a execução de `free_ed_list()`, o que dificultou a análise durante testes mais complexos.

Estas falhas evidenciaram a importância de uma abordagem mais defensiva, reutilizável e validada desde o início.

Lições Aprendidas e Melhoria Contínua

Este projeto permitiu consolidar conceitos fundamentais como gestão de memória, manipulação de listas ligadas e organização modular do código. Além disso, destacou a importância de:

- Projetar funções com **responsabilidades únicas**
- **Validar dados de entrada** antes de alterar estruturas
- Reutilizar funções auxiliares e **evitar duplicação de lógica**
- Aplicar ferramentas como o **Doxygen**, tanto para documentação como para apoio à arquitetura do sistema

Propostas de Melhoria Futuras

Durante o desenvolvimento e posterior análise do projeto, surgiram ideias e considerações que, embora não tenham sido implementadas nesta fase, poderão representar melhorias relevantes no futuro:

Utilização de apontador tail

A dada altura chegou a ser implementado um **apontador tail** na estrutura da lista ligada, com o objetivo de acelerar a inserção no fim da lista, evitando a travessia completa da estrutura. Esta ideia, embora **não tenha sido abordada nas aulas**, surgiu de forma espontânea como tentativa de otimização.

Contudo, após testes e análise crítica, concluiu-se que a inserção era feita de forma ordenada, pelo que a inclusão de tail não trazia vantagens reais neste contexto. Além disso, a sua manutenção traria complexidade desnecessária, especialmente durante remoções. Por esse motivo, **decidiu-se remover esta funcionalidade**, mas mantê-la como proposta para cenários com inserção sequencial.

Reestruturação de funções com lógica repetida

A função `remove_ed()` contém lógica de pesquisa duplicada, semelhante à usada em `find_ed()`. Uma futura melhoria poderá passar por criar uma função auxiliar

`find_previous_ed()` ou adaptar `remove_ed()` para reutilizar a função de procura já existente, reduzindo redundância e aumentando a clareza do código.

Generalização com apontadores para função

A função `find_ed()` poderia ser transformada numa função genérica que recebe um **apontador para função de comparação**, permitindo utilizá-la com diferentes critérios (por exemplo, por coordenadas, por frequência, etc.). Esta abordagem aumentaria a reutilização de código e a flexibilidade do sistema.

Capítulo 6 – Conclusão

O presente relatório descreveu o desenvolvimento da Fase 1 do trabalho prático de Estruturas de Dados Avançadas, cuja principal meta consistiu na aplicação de algoritmos de gestão de listas ligadas, recorrendo a estruturas dinâmicas, em linguagem C.

Durante o processo, foram implementadas funcionalidades essenciais como a criação, inserção ordenada, pesquisa, remoção e libertação de memória de uma lista ligada, bem como a deteção de conflitos entre antenas com frequências iguais, originando a criação de efeitos nefastos. A organização modular do código e a separação de responsabilidades entre funções permitiram um desenvolvimento mais estruturado, facilitando a legibilidade e manutenção do sistema.

Apesar de algumas dificuldades técnicas encontradas — nomeadamente na propagação dos efeitos e na gestão de apontadores em listas ordenadas —, todas as funcionalidades principais foram cumpridas. O processo revelou também a importância da validação de dados à entrada e da reutilização de funções, aspetos que surgiram como oportunidades de melhoria numa análise posterior.

Foram ainda consideradas, testadas e descartadas algumas abordagens alternativas, como a utilização de um apontador *tail*, que embora válida noutros contextos, não se revelou benéfica nesta fase, dado o tipo de inserção requerida. Esta reflexão contribuiu para uma maior consciência técnica e capacidade de avaliação crítica.

De forma geral, este trabalho permitiu consolidar conhecimentos fundamentais sobre estruturas dinâmicas, reforçar boas práticas de programação em C e desenvolver autonomia na análise e resolução de problemas computacionais. As lições aprendidas nesta fase constituem uma base sólida para a continuidade do projeto em fases futuras, com maior grau de complexidade.

Referências

- Damas, L. (2002). *Programação em C*. FCA Editora de Informática.
- Horowitz, E., Sahni, S., & Anderson-Freed, S. (2009). *Fundamentos de Estruturas de Dados em C*. Bookman.
- Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- Weiss, M. A. (2011). *Data Structures and Algorithm Analysis in C* (3rd ed.). Addison-Wesley.